



교육 콘텐츠 관리 플랫폼

교육 콘텐츠 웹 뷰어 및 AI 관리 시스템

학과 : AI융합교육학과
과목 : AI기술융합프로젝트
지도 : 차가을 교수님
발표 : 박수연/박희용

CONTENTS

PART 01 · 프로젝트 개요

추진 배경과 목적 · 시스템 구성 · 개발 방식
과 역할 분담

PART 02 · 시스템 아키텍처

전체 구조도 · 데이터 저장 3원화 · 핵심 위
크플로 · 보안 구조

PART 03 · 주요 기능

수강생 뷰어 · 강사 대시보드 · 실시간 설문
· AI 기능 4종

PART 04 · AI 기능 상세 · 기술 스택

FastAPI · LangChain · LangGraph ·
GPT-4o — 호출 구조와 체인 설계

PART 05 · 설계 검증 · 비교 사례

반복된 실패가 설계를 바꾼 과정 — AS-IS
/ TO-BE 비교

PART 01

프로젝트 개요

추진 배경과 목적 · 시스템 구성 · 개발 방식과 역할 분담

프로젝트 배경 및 목적

교육 콘텐츠의 안전한 공유와 생성형 AI 활용을 하나로 묶는 통합 플랫폼

추진 배경

다양한 교육 운영 업무

AX·DX 교육과 일본 IT 해외취업 과정을 운영하며
교육관리 · 취업지원 · 교육 콘텐츠 관리 · 홍보마케팅 제작 등 다양한 업무를 수행

통합 플랫폼의 필요성

교육 콘텐츠를 안전하게 공유하고, 생성형 AI로
다양한 형태의 교육 자료를 관리할 수 있는 별도
플랫폼 구축 요구.
향후 취업자료 기반의 분석도구로 확장.

프로젝트 목적

1

안전한 콘텐츠 공유

강의 교안(PPT)을 웹 기반으로 변환.
수강생은 편집·다운로드 없이 목차와 함께 열람.
강사는 웹상에서 슬라이드 쇼 형태로 자료를
공개하며 강의진행 중 실시간으로 질문
답변을 진행하고 기록.

2

AI 기반 강의 지원

시험문제 생성 · 설문 의견 요약 · 가상 학생질문 · 강의자료 평가를 생성형 AI로 자동화

시스템 구성 및 개발 방식

기본 기능은 별도 서버·DB를 새로 구축하지 않고 Google Workspace 자원을 최대한 활용한 경량 구성.

전체 : GAS 프로젝트 3개 + AI 전용 서버 1개 + Database 서버 1개

publish-engine

GAS 라이브러리

슬라이드 변환·웹게시·목차 추출·DB 기록을 담당하는 공용 "엔진" - 2개 시스템이 공유

system-a-viewer

웹앱 · 전체 공개

수강생 뷰어 — 로그인 불필요, 키워드로 구분하여 접속, 목차+슬라이드 열람 · 설문 응답

system-b-dashboard

웹앱 · 조직 내부

강사 대시보드 — 배포/수정/삭제, 실시간 설문 출제, AI 기능 4종 호출

ai-server

Cloud Run · Python

AI 전용 백엔드 — FastAPI 엔드포인트 4개, LangChain/LangGraph로 GPT-4o 호출

개발 방식 · 환경

서버리스 구성 Google Workspace(Drive·Slides·Sheets) 자원 활용, 배포·접근제어는 구글 계정 체계에 위임

로컬 개발 clasp(로컬↔Apps Script 동기화) + Claude Code 기반 개발

형상 관리 5개 서브프로젝트(GAS 4 + ai-server)를 한 저장소로 관리

자동 배포 로컬 프로젝트 → GitHub Push → Google Cloud Run 에 자동 빌드, 배포하는 원클릭 배포 환경 구현

개발 업무 분배

박수연

아키텍처 설계.
라이브러리 및 API 개발.
공통 모듈·게시엔진 개발,
배포 관리

박희용

아키텍처 설계. 요구사항 정의·기능 우선순위와 운영 시나리오 설계, 데이터구조 설계. 뷰어 화면 구성, 사용자 테스트

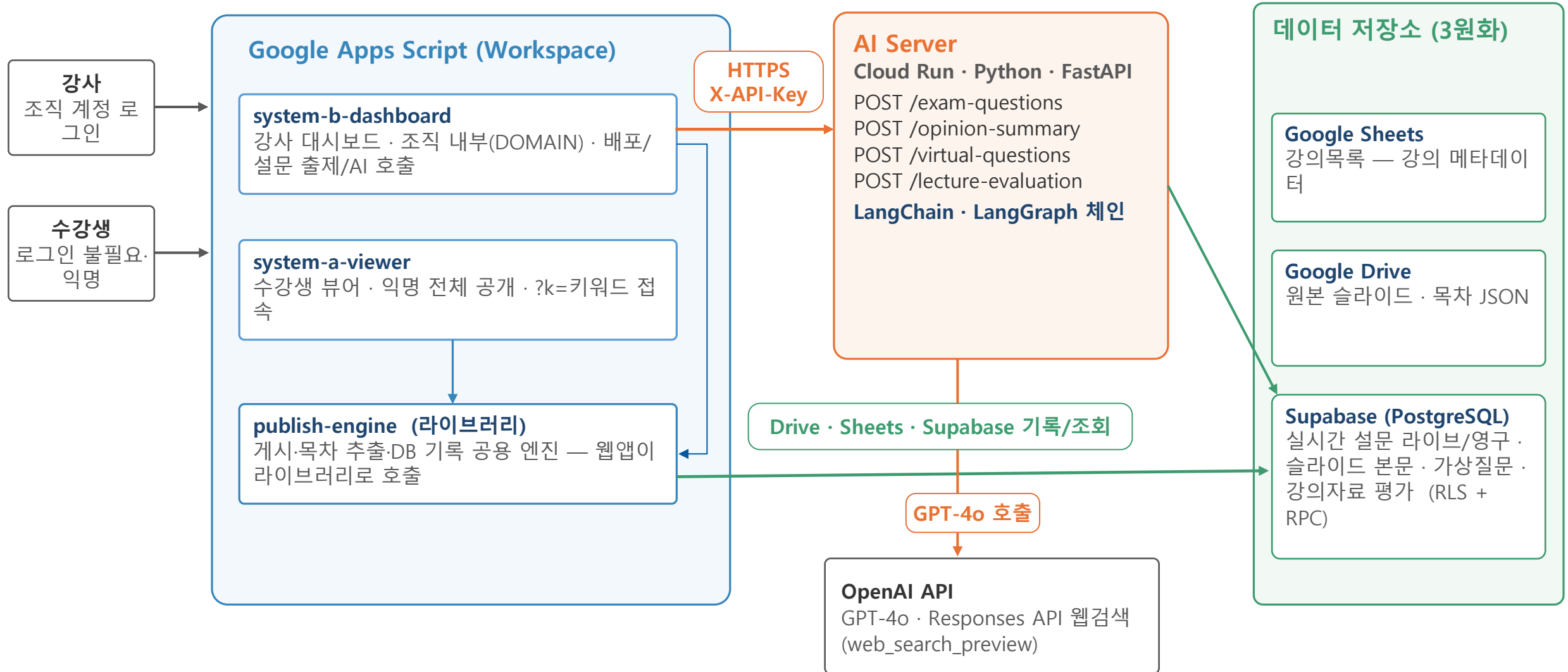
PART 02

시스템 아키텍처

전체 구조도 · 데이터 저장 3원화 · 배포/설문 워크플로 · 보안 구조

전체 시스템 아키텍처

GAS 웹앱 2종 + 공용 라이브러리 + Cloud Run AI 서버 + 저장소 3종 — 전 구간 HTTPS



데이터 저장 구조 — 3원화 전략

데이터의 성격에 따라 저장소를 분리 — 관리 편의성(Sheets) · 응답 속도(Drive JSON) · 트랜잭션(Supabase)

Google Sheets 강의목록 시트

저장 데이터

강의자료의 메타데이터 — 키워드(고유키) · 강의제목 · 슬라이드ID · 게시URL · 목차JSON 링크 · 최종수정시간

성격

키워드 기준 행 단위 upsert

선택 이유

강사가 직접 열어볼 수 있는 투명한 저장소 — 별도 DB 없이 메타 관리

Google Drive 목차데이터 폴더 · 키워드.json

저장 데이터

목차(TOC) 스냅샷 — 뷰어가 읽는 유일한 데이터 소스

성격

강의 교체 시 덮어쓰기(스냅샷) — 뷰어 URL은 영구 고정

선택 이유

파일 1개 읽기로 뷰어 렌더링 완결 — 단순하고 빠른 데이터 통로

Supabase (PostgreSQL) PostgREST REST API

저장 데이터

실시간 설문(라이브+영구) · 슬라이드 본문 · 가상질문 페르소나/결과 · 강의자료 평가

성격

AI 기능의 근거 데이터이자 결과 캐시 — RPC 2종으로 경쟁상태 원자화

선택 이유

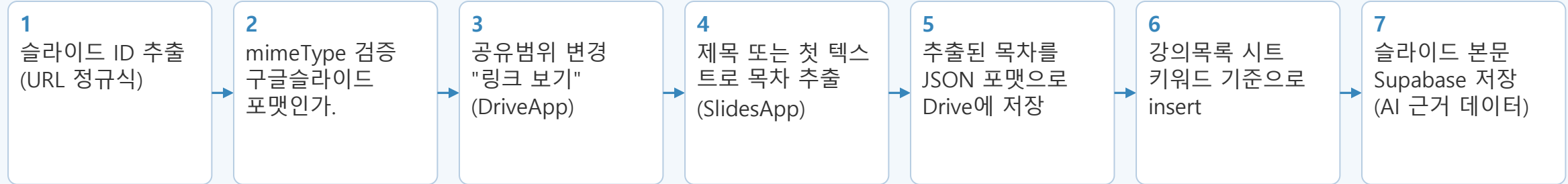
GAS·Python 양쪽에서 REST로 동일 접근 · RLS 내장

핵심 워크플로 ① — 강의 배포

강사의 [배포] 클릭 한 번으로 웹게시 준비가 완료 — publishLecture() 내부 7단계 자동 처리



publishLecture() 내부 처리 순서



URL 영구 고정

같은 키워드 재배포 시 뷰어 URL은 그대로,
내용만 갱신 — 학생에게 다시 공유할 필요 없음

경량 수정 경로

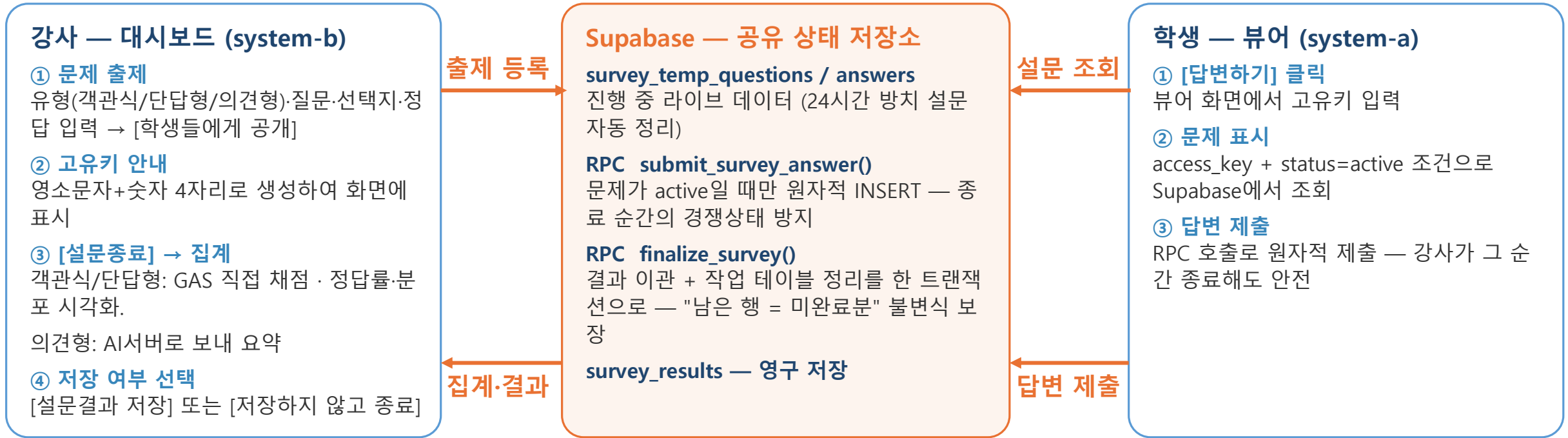
슬라이드 재추출 없이 강의정보만 갱신하
는 별도 경로 제공

안전한 배포 취소

배포 취소시 같은 슬라이드를 다른 키워드
가 참조 중이면 공유 설정은 유지, 관련 데
이터만 정리

핵심 워크플로 ② — 실시간 설문

서로 다른 GAS 배포(대시보드 ↔ 뷰어)가 Supabase를 공유 상태 저장소로 사용해 실시간 통신



문제 — 배포 간 상태 공유 불가

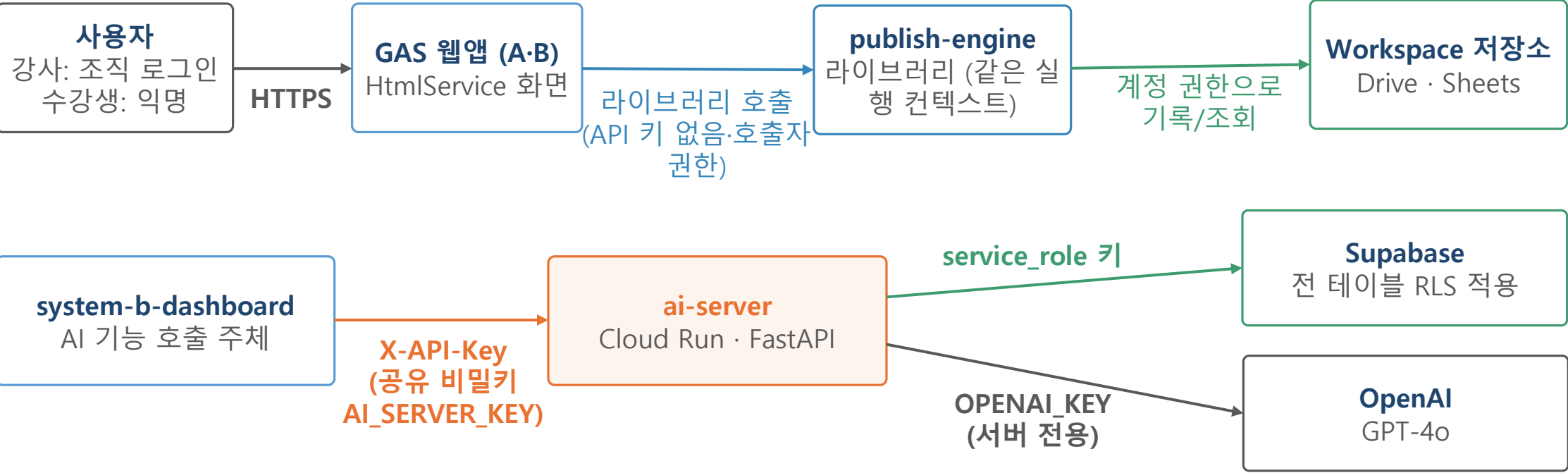
대시보드와 뷰어는 서로 다른 GAS 배포라 CacheService를 공유할 수 없음 → 진행 중 설문 상태를 주고받을 방법이 필요

해결 — DB를 통한 간접 통신 + RPC 원자화

라이브 상태를 Supabase 작업 테이블에 저장해 두 웹앱이 간접 통신, "조회 후 쓰기" 경쟁상태는 RPC 트랜잭션으로 원자화.
RPC(Remote Procedure Call, 원격 프로시저 호출)

보안 · 인증 구조

계층 간 인증 방식을 분리 — 모델 제공자 키는 ai-server에만 보관



모델 키 집중 보관

OpenAI 키는 Cloud Run 환경변수에만 존재 — GAS 스크립트 속성에는 없음. 키 유출 면적 최소화

설정 중앙화

모든 ID·URL·키는 Config.js의 PropertiesService에 집중, 민감 키는 getSecret() 화이트리스트 검증 후에만 반환

원본 파일 비노출

수강생에게는 웹게시 URL만 제공, 원본 링크 미노출 → 편집·다운로드 차단

PART 03

주요 기능

수강생 뷰어 · 강사 대시보드 · 실시간 설문 · AI 기능

주요 기능 ① — 수강생 뷰어 (System A)

로그인 없이 링크 하나로 열람 — 편집·다운로드는 원천 차단

키워드 URL 접속

?k=키워드 링크만으로 접속 — 로그인·계정 불필요, 외부 수강생도 즉시 이용

목차 + 슬라이드 열람

좌측 목차 사이드바(검색 지원) + 우측 슬라이드 embed, 목차 클릭 시 해당 슬라이드로 이동

원본 보호

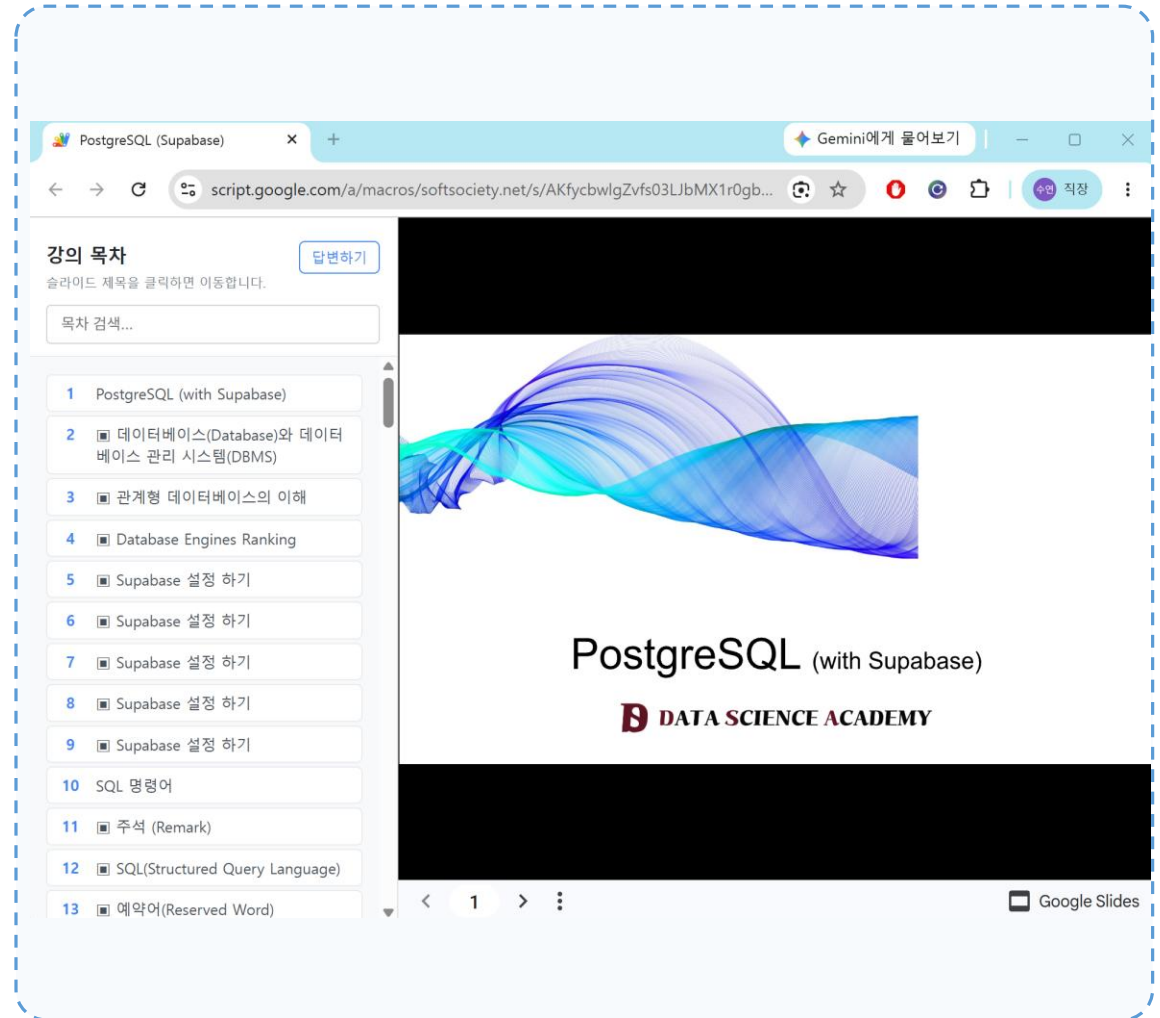
웹게시 화면만 노출하고 원본 파일 링크는 제공하지 않음 → 편집·다운로드 차단

실시간 설문 참여

[답변하기]에서 고유키 입력 → 진행 중 설문에 익명으로 응답

URL 영구 고정

같은 키워드로 재배포하면 내용만 갱신 — 게시판에 공유한 링크가 계속 유효



주요 기능 ② — 강사 대시보드 (System B)

조직 내부 전용 — 배포부터 AI 기능까지 강의 운영의 단일 진입점

- 원클릭 배포**

키워드·제목·슬라이드 URL 입력 후 [배포] → 변환·웹게시·목차 추출·DB 기록 자동 처리
- 강의 목록 관리**

배포된 강의 목록 표 — 뷰어 URL 복사·뷰어 열기·수정·삭제, 제목만 경량 수정 지원
- 실시간 설문 출제**

문제 출제 → 고유키 공개 → 종료 시 자동 채점·정답률·분포 집계 → 저장 여부 선택
- AI 기능 4종 호출**

시험문제 생성 · 의견 요약 · 가상질문 · 강의자료 평가 — AI_MODE 스크립트 속성으로 ON/OFF 제어
- 접근 제어**

조직 내부(DOMAIN) 접근 · 접속자 본인 권한으로 실행, 도움말 페이지 내장

강의 콘텐츠 관리

사용법은 도움말을 참고하세요.

도움말

+ 새 강의 추가하기

2개 강의

강의 제목으로 검색

번호	키워드	강의 제목	최종 배포	뷰어 URL	기능	AI Mode : ON
1	sample	강의자료 샘플 (내용없습니다)	2026-08-01 17:00	🔗	🔗	배포 수정 실시간 설문 AI 시험문제 생성 AI 가상질문 생성 AI 강의자료 평가
2	postgresql	PostgreSQL (Supabase)	2026-08-01 14:56	🔗	🔗	배포 수정 실시간 설문 AI 시험문제 생성 AI 가상질문 생성 AI 강의자료 평가

새 강의 추가

키워드

예: react-basic

뷰어 주소(키워드)에 고정으로 쓰입니다. 이미 있는 키워드는 등록할 수 없습니다(수정은 목록의 "배포 수정" 이용).

강의 제목

예: 리액트 기초

슬라이드 편집 URL

https://docs.google.com/presentation/d/...

공개하고자 하는 구글슬라이드 파일을 열어 주소창에 표시되는 URL을 복사해서 붙여넣기 하세요.

목차 추출 방식

☒ 제목만 추출(제목 개체가 있는 슬라이드만)

☐ 첫 번째 문자열 사용(모든 슬라이드의 첫 텍스트)

취소

배포

실시간 설문 출제

질문

문제유형

☒ 객관식

☐ 단답형

☐ 의견형

선택보기 (2개 이상)

1. 보기 1

×

2. 보기 2

×

+ 보기 추가

정답

정답 보기 번호(여러 개면 쉼표로 구분, 예: 1,3)

학생 제출도 번호로 처리되어 이 번호와 일치하면 정답 처리됩니다.

취소

학생들에게 공개

주요 기능 ③ — 실시간 설문

수업 중 즉석 출제와 실시간 집계 — 의견형은 AI 요약까지

3가지 문제 유형

객관식 · 단답형 · 의견형 — 유형별 입력 폼과 집계 방식 자동 전환

고유키 참여

혼동 문자를 뺀 4자리 고유키(o.l·0.1 제외) — 학생은 키 입력만으로 참여

안전한 동시 제출

강사가 종료하는 순간의 제출도 유실·중복 없이 처리

자동 집계

종료 시 정답률·응답 분포 즉시 계산, 의견형은 GPT-4o 요약(실패 시 원문 그대로 표시)

저장 선택권

[설문결과 저장] 시 트랜잭션으로 영구 보관, [저장하지 않고 종료] 시 흔적 없이 정리

The screenshot displays the '실시간 설문 출제' (Real-time Survey Question Creation) interface. It includes a question input field with a sample question about SQL queries, a question type selector (radio buttons for 객관식, 단답형, 의견형), and a list of selected questions (1. SELECT ALL student; 2. SELECT * FROM student; 3. GET * student; 4. SHOW TABLE student;). Below this is the '설문 결과' (Survey Results) section, showing a list of questions and their corresponding answers, with a '보기 추가' (Add View) button. The '수업 중 가장 어려운' (Most difficult during class) section shows a list of questions and their corresponding answers, with a '보기 추가' (Add View) button. The 'AI 요약' (AI Summary) section shows a summary of the survey results, including a table of results and a '결과를 저장하지 않고 종료' (End without saving results) button. The '설문 진행 중' (Survey in progress) section shows a progress bar and a '설문 종료' (End Survey) button. The '설문 결과' (Survey Results) section shows a table of results, including a table of results and a '결과를 저장하지 않고 종료' (End without saving results) button.

실시간 설문 출제

질문
student 테이블의 모든 데이터를 조회하는 SQL문으로 올바른 것은 무엇인가?

문제유형
☒ 객관식
☐ 단답형
☐ 의견형

선택보기 (2개 이상)

1. SELECT ALL student;
2. SELECT * FROM student;
3. GET * student;
4. SHOW TABLE student;

설문 결과

수업 중 가장 어려운

AI 요약

다수의 학생들이 데 있는 것으로 보입니
온 부분으로 언급되
일부 학생들이 지적했습니다. 전반적으로 수업에서 다루는 기술적
들이 학생들에게 도전적인 과제로 작용하고 있는 분위기입니다.

전체 답변

1. 실행환경 설정이 어려워요
2. 테이블 설계
3. 데이터 구조를 설계하는것이 어렵습니다
4. 명령어 외타난것 잡기가 어려워요
5. 용어나 개념 이해가 가장 어렵습니다.
6. 테이블을 어떻게 만들어야할지 모르겠어요

설문 진행 중

학생들에게 아래 고유키를 알려주세요

FT4X

경과 01:36

설문 종료

설문 결과

데이터베이스 관리 시스템(DBMS)의 주요 기능이 아닌 것은 무엇인가
요?

응답 6건 · 정답률 50%

3. 데이터의 중복성 증가 ✓	3건 (50%)
4. 데이터의 보안성 제공	2건 (33.3%)
2. 데이터의 무결성 유지	1건 (16.7%)

결과를 저장하지 않고 종료

설문결과 저장

주요 기능 ④ — AI 기능 4종

전부 OpenAI GPT-4o 단일 제공자로 통일 — 전용 백엔드(ai-server)를 통해서만 호출

시험문제 생성

슬라이드 본문 + 가상질문을 근거로 유형(객관식/단답형/서술형) × 난이도(초/중/고) 조합 문제 생성.

강사의 평가시험 출제에 사용 목적.

가상질문 생성

각 수준별 학생 페르소나가 슬라이드를 순서대로 읽으며 "막히는 지점"의 질문을 생성

설문 의견 요약

수업 중 실시간으로 학생들에게 의견을 묻는 설문을 할때 학생 답변을 요약하여 수강생들의 현재 상태를 즉시 파악.

강의자료 평가

강의자료의 구조·구성 검토 + 수업중진행된 실시간 설문 결과 + 학생 페르소나의 가상 질문 + 웹검색 기반 최신성 검토를 산문형 리포트로 제공.
강의자료의 개선에 도움이 되도록 한다.

가상질문 생성 - 학생 선택

질문을 생성하거나 확인할 학생을 선택하세요. 이미 생성된 결과가 있으면 바로 보여줍니다.

학생1(초보자)

이 분야 사전지식이 없고 용어·개념이 생소한 학습자

학생2(비전공자)

비전공이지만 관심이 많고 그럭저럭 수업을 따라가는 학습자

학생3(전공자)

관련 전공자이지만 실무 경험은 없고 이

학생4(실무경험자)

짧지만 관련 실무나 프로젝트 경험이 있

강의자료 평가

슬라이드 + 실제/가상 반응 기반

- 슬라이드 본문: 전체 143장 텍스트

- 실시간 설문 결과: 6건 참고함

- 가상 학생 질문: 학생1(초보자) 30개, 학생2(비전공자) 22개, 학생3(전공자) 16개, 학생4(실무경험자) 20개 참고함

구조 검토

슬라이드의 구성은 전반적으로 PostgreSQL과 Supabase의 개념을 소개하고, 데이터베이스의 기본 개념부터 고급 기능까지 체계적으로 설명하고 있습니다. 그러나 몇 가지 개선할 점이 있습니다:

1. **용어 정의의 순서**: 일부 슬라이드에서는 용어가 정의 없이 먼저 등장합니다. 예를 들어, "데이터베이스"와 "DBMS"의 정의가 슬라이드 2에서 나오지만, 그 전에 이 용어들이 이미 사용되고 있습니다.

2. **슬라이드당 정보량**: 몇몇 슬라이드(예: 슬라이드 2, 3, 6)는 정보량이 많아 학습자가 한 번에 이해하기 어려울 수 있습니다. 정보량이 많은 슬라이드는 두 개 이상의 슬라이드로 나누어 설명하는 것이 좋습니다.

3. **예시 부족**: 개념 설명 후 예시가 부족한 경우가 있습니다. 특히, 관계형 데이터베이스의 개념 설명 후 실제 예시가 추가되면 이해에 도움이 될 것입니다.

닫기

다시 평가

DATA SCIENCE ACADEMY

16

PART 04

AI 기능 상세 · 기술 스택

FastAPI · LangChain · LangGraph · OpenAI GPT-4o — 호출 구조와 체인 설계

AI 기술 스택 구성

호출 게이트부터 모델·데이터·배포까지 — 계층별 역할과 선택 이유

호출

GAS UrlFetchApp

system-b 대시보드에서만 호출 · X-API-Key 공유 비밀키 인증 · AI_MODE ON/OFF 게이트

API 서버

FastAPI + Uvicorn + Pydantic

Cloud Run 컨테이너 · Uvicorn: 비동기 ASGI 서버로 앱 구동 / Pydantic: 요청·응답 데이터 검증 및 API 문서 자동화

오케스트레이션

LangChain (langchain-openai)

ChatOpenAI 공용 프로바이더

에이전트

LangGraph

StateGraph 상태 유지형 순차 루프 — 가상질문 생성의 "구간 읽기 + 컨텍스트 누적 + 조건부 반복"을 명시적으로 표현

LLM 모델

OpenAI GPT-4o

AI 4개 기능 전부 단일 제공자로 통일 · Responses API 내장 웹검색(web_search_preview) 활용

데이터

Supabase PostgREST

슬라이드 본문·설문 결과·페르소나를 REST로 직접 조회, 결과는 캐시 테이블에 upsert

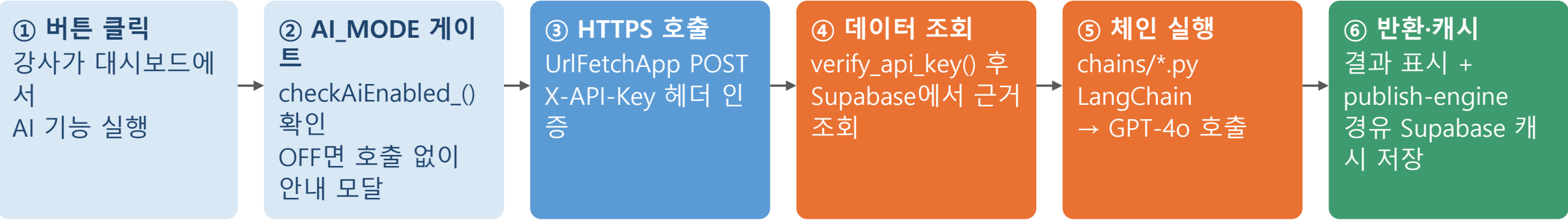
배포

Cloud Build → Artifact Registry → Cloud Run

GitHub main 브랜치 push만으로 빌드~배포 자동화 (cloudbuild.yaml, 모노레포 하위 디렉터리 빌드)

AI 기능 공통 호출 구조

4개 기능이 동일한 파이프라인을 공유 — 게이트 확인부터 결과 캐시까지

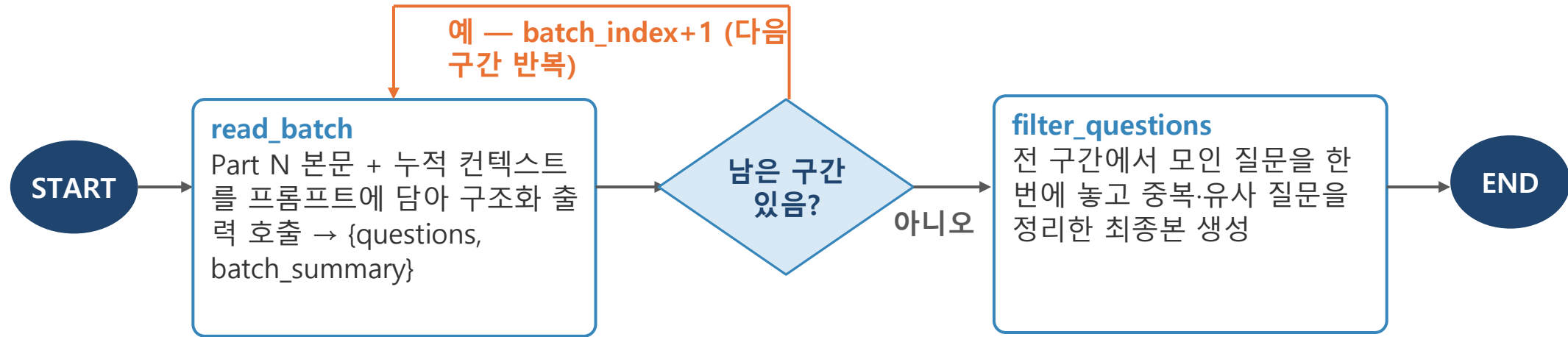


엔드포인트별 체인 구성

기능	엔드포인트	LangChain 구조	캐시 정책
시험문제 생성	POST /exam-questions	구조화 출력 단발 체인 — 유형×난이도별 프롬프트 + 가상질문 참고자료 주입	캐시 없음 (매번 새로 생성)
설문 의견 요약	POST /opinion-summary	단발 체인 — grounding 규칙(숫자 금지·4단계 비율 기준) 프롬프트	캐시 없음 (설문 1회성)
가상질문 생성	POST /virtual-questions	LangGraph StateGraph — read_batch 루프 + filter_questions	키워드+페르소나당 최신 1건
강의자료 평가	POST /lecture-evaluation	구조화 출력 1회 + 웹검색 2단계(조사→재작성) 격리 호출	키워드당 최신 1건

LangGraph 기반 가상질문 생성 그래프

"구간별 읽기 → 컨텍스트 누적 → 조건부 반복 → 최종 필터링"을 StateGraph로 명시적으로 표현



컨텍스트 누적 전략

슬라이드 50장 이하: 지나온 구간 원문을 그대로 누적
50장 초과: 원문 대신 구간 요약 (batch_summary)만 누적 — 별도 요약 호출 없이 토큰 절약

"질문 없음"도 정상 결과

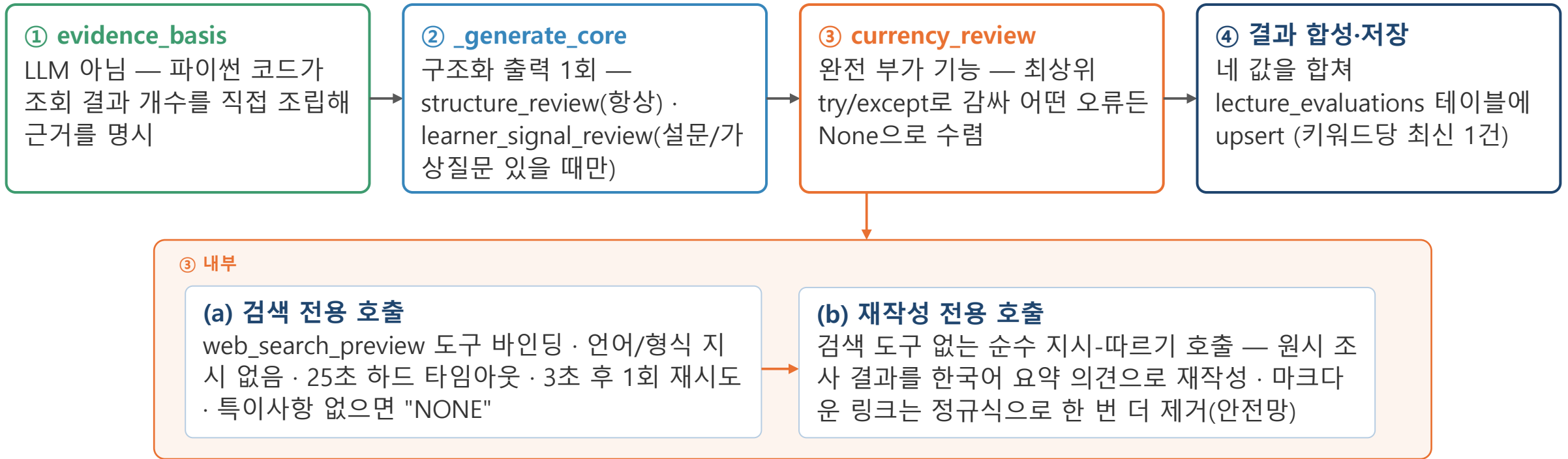
의미 있는 내용이 없거나 이미 다룬 내용과 겹치면 억지로 생성하지 않음

페르소나 = 데이터

4종 페르소나(초심자·비전공·전공이론파·실무경험자)를 virtual_question_personas 테이블로 관리 — 추가·수정에 코드 변경 불필요

강의자료 평가 — 4단계 파이프라인

핵심 평가와 부가 기능(웹검색)을 구조적으로 격리 — AI의 웹검색이 실패해도 DB의 자료를 근거로 평가



입력 데이터 (Supabase 조회)

슬라이드 본문(slide_contents) + 실시간 설문 결과 (survey_results) + 페르소나별 가상질문(virtual_questions) — 학습자 신호까지 근거로 활용

실패 격리 설계

웹검색 쪽에서 아직 겪어보지 못한 새 오류가 나도 예외를 던지지 않고 항상 None으로 수렴 → 핵심 평가는 유지되고 해당 섹션만 조용히 생략 — 개별 패치 없이 자동 처리

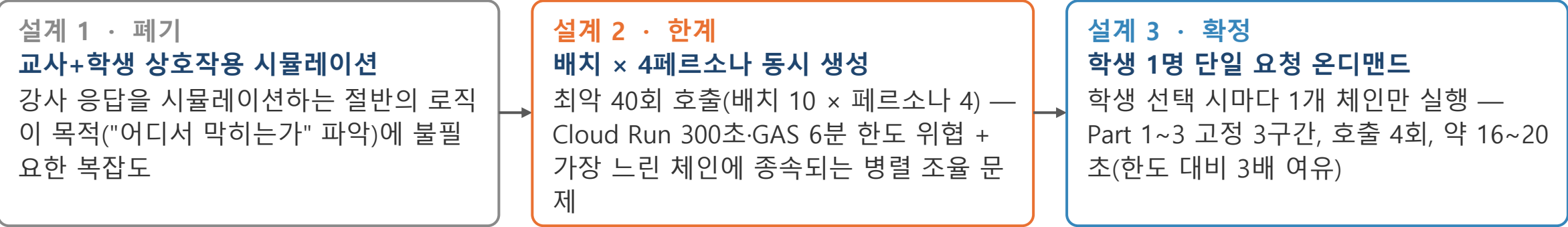
PART 05

설계 검증 · 비교 사례

반복된 실패가 설계를 바꾼 과정 — 개발 기록 기반 AS-IS / TO-BE 비교

사례 ① — 가상질문 생성: 4페르소나 동시 → 1명 온디맨드

요구사항에서 아키텍처까지 — 설계가 세 번 바뀐 과정과 그 이유



구분	AS-IS — 4페르소나 동시 생성	TO-BE — 1페르소나 온디맨드
요청 단위	버튼 1회 → 4개 체인 동시 실행	학생 선택 시마다 1개 체인 실행
호출 횟수 (최악)	배치 10개 × 페르소나 4명 = 40회	배치 3회 + 필터링 1회 = 4회
배치 분할	전체의 10% 동적 분할 (3~15장 clamp)	초/중/후반 3구간(Part 1/2/3) 고정
소요 시간·위험	병렬 조율 필요 — 가장 느린 체인에 종속, 타임아웃 한도 위험	약 16~20초 (60초 한도 대비 3배 여유) + 캐시 시 즉시 반환

관점의 전환 병렬 처리를 최적화하는 대신, 병렬이 필요 없도록 요청 단위 자체를 쪼갬 · 페르소나 정의는 코드 → 테이블(FK)로 이동해 추가 시 코드 변경 불필요

사례 ② — 웹검색: 단일 호출 vs 2단계 분리

1단계 — 인프라 실패

TPM 한도 초과로 RateLimitError(429), 이어서 응답 파싱 AttributeError로 전체 요청 500 — 결과 자체가 없음

원인: 원문 재전송(사례 ③) + content가 리스트로 오는 경우 미처리

2단계 — 스타일 미준수

호출은 성공했지만 영어 응답 · 글머리 기호 나열 · 마크다운 링크 노출 — 스타일 지시가 지켜지지 않음

"As of August 2, 2026, the latest stable release of PostgreSQL is version 18..."

3단계 — 분리 후 성공

검색 전용 + 재작성 전용으로 분리 — 한국어 요약 톤, 링크 없음, 강의 주제와 연결된 최신 정보, 실행시간도 단축

"PostgreSQL의 최신 안정 버전은 18.4로 ..." (한국어 요약 의견)

원인 분석 — 도구 바인딩이 형식 지시를 밀어냄

검색 도구가 바인딩된 호출에서는 "찾은 내용을 보고하기"라는 도구 본연의 동작이 출력 형식·언어 지시보다 우선되는 경향이 실측으로 반복 확인됨

해결 — 역할을 분리한 2단계 파이프라인

① 검색 전용 호출

도구 바인딩 · 형식 지시 없음
"있는 그대로" 조사만

② 재작성 전용 호출

도구 없음 · 순수 지시 따르기
한국어 2~4문장 요약으로

설계 원칙 한 호출에 서로 다른 성격의 지시(도구 사용 vs 출력 형식)를 함께 부여하지 않는다 — "한 가지만 잘하면 되는" 호출로 나누면 지시 순응도가 크게 오른다

사례 ③ — 원본 재전송 vs 파생 요약 재사용 (토큰 실측)

강의자료 평가 1회 실행 안의 두 번째 LLM 호출(웹검색)에 무엇을 입력할 것인가
TPM은 요청단위가 아닌 분단위 누적. 1분안에 2개의 요청이 이 한계를 넘음 (근거 : <https://www.codewords.ai/blog/openai-api-rate-limits>)
429 : 한도초과시 HTTP 오류. 이 메시지의 내용 기준으로 토큰량 확인.

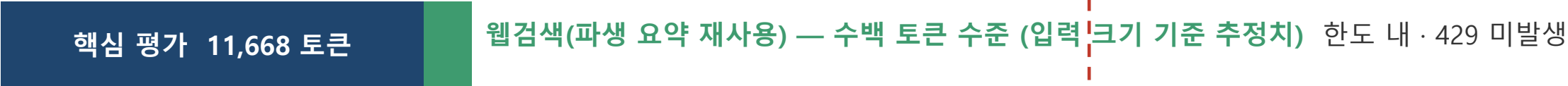
AS-IS — 슬라이드 전체 원문(143장)을 다시 전송

조직 TPM 한도 30,000



합계 30,513 토큰 > 30,000 TPM → RateLimitError(429) — "1.026초 후 재시도" 안내와 함께 거부

TO-BE — 웹 검색을 위해 2차 요청 시 원문이 아닌 핵심 평가가 이미 만든 요약(structure_review)을 재사용



웹검색 입력 크기
AS-IS 슬라이드 전체 원문 (~143장, 11,000자+)
TO-BE 구조 검토 요약 1건 (수백 토큰)

레이트리밋(429)
AS-IS 발생 — TPM 한도 초과
TO-BE 미발생

전체 실행시간
AS-IS 약 40초 (재시도 대기 포함)
TO-BE 약 20초. 이전보다 단축 (실측)
슬라이드가 더 늘어나도 안전.

시사점 강의자료 원본을 평가하고, 최신정보를 웹에서 검색할때 이미 생성된 파생 요약데이터의 재사용이 토큰·지연·안정성 모두에서 유리

사례 ③ 보강 — 토큰 숫자는 어디서 나온 값인가

앞 페이지의 30,513 토큰과 1.026초는 추정이 아니라 OpenAI가 알려준 값

토큰 사용량은 세 곳에서 확인된다

① 호출이 성공하면

응답 안에 사용량이 함께 온다

보낸 양 · 받은 양 · 합계
실제로 처리된 양 = 요금이 매겨지는 값
실패한 호출에는 없다

② 호출이 거부되면 (429)

에러 메시지가 숫자를 알려준다

한도 · 이미 쓴 양 · 이번 요청량 · 대기 시간
OpenAI가 직접 센 값
앞 페이지 숫자의 출처가 바로 여기

③ 성공이든 실패든

응답 헤더에 남은 여유가 온다

이번 분에 아직 쓸 수 있는 토큰 양
터지기 전에 미리 볼 수 있는 유일한 값
본문이 아니라 헤더라 따로 읽어야 한다

429 메시지 하나로 앞 페이지 숫자가 전부 검산된다

※ 아래는 실측 수치를 429 응답 형식에 대입한 재구성. 티어에 따라 다름.
OpenAI의 티어는 "누적 결제액"과 "첫 결제 이후 경과일" 두 조건을 모두 충족해야 올라간다.

1분 한도

30,000

이미 쓴 양 (핵심 평가)

11,668

이번 요청 (웹검색)

18,845

한도 초과분

513

1분에 30,000 = 1초에 500씩 회복된다. 따라서 초과분 513 ÷ 500 = 1.026초 — 에러가 알려준 대기 시간과 일치
→ 숫자를 우리가 짜맞춘 것이 아니라, OpenAI가 센 값끼리 맞아떨어진다는 뜻이다.

확인 ① 이번 429의 원인은 '보낸 양' 하나뿐

max_tokens = 답변을 최대 몇 토큰까지 만들지 미리 정해두는 상한값.
우리 코드는 이 값을 지정하지 않아 출력분이 미리 예약되지 않는다.
따라서 원인은 슬라이드 143장을 다시 보낸 것 하나로 확정된다.

확인 ② 제한을 걸면 오히려 불리해질 수 있다

지정하면 실제 출력이 아니라 '설정한 숫자'가 요청 시점에 미리 차감된다.
넉넉히 4,000을 걸고 8개를 동시에 돌리면 그것만으로 32,000 > 30,000.
제한은 출력이 짧고 예측 가능한 호출에만 좁게 거는 편이 안전하다.

시사점 성공은 응답에서 · 실패는 에러 메시지에서 · 여유는 헤더에서. 세 곳을 모두 로그에 남겨야 다음 개선의 근거가 남는다

종합 인사이드 — 세 사례를 관통하는 설계 원칙

개발 과정의 반복된 실패와 실측이 도출한 공통 원칙

1

파라미터 조정보다 아키텍처 조정

레이트리미트를 재시도로 완화하기보다 입력 데이터 크기 자체를 줄이는 것이, 병렬 호출을 최적화하기보다 요청 단위를 쪼개는 것이 더 근본적인 해결이었다.

2

한 호출에 한 가지 책임

검색과 스타일 준수를 한 호출에 같이 시키면 지시 순응도가 떨어진다. 역할을 분리하면 각 호출이 단순해지고 결과 품질이 올라간다.

3

가변적인 것은 데이터로, 고정적인 것은 코드로

페르소나 정의처럼 바뀔 가능성이 높은 부분은 테이블(FK)로 옮기고, 그래프 구조처럼 안정적인 로직만 코드로 남긴다.

4

실측 없이 확답하지 않는다

호출 횟수·지연시간·토큰량 모두 추정에 그치지 않고 실기기 테스트로 검증한 뒤에야 다음 설계 단계로 넘어갔다.



Thank you

